

Perfect ✅ — let's continue with **Lecture 2: "Programming Review and Introduction to Software Design."**

I'll explain **each section** (like a teacher) with **80% focus on understanding** and only **20% technical detail**, so it feels clear and practical.

Lecture 2: Programming Review and Introduction to Software Design

1. Purpose of the Lecture

This lecture helps you:

- Review good programming habits
- Learn **how to document and describe functions** properly
- Understand **what software design is**
- Know the **goals and qualities** of a good software design

Think of it as:

"How to write clean, understandable, and well-planned code before you even start coding."

Part 1 — Documenting Functions

2. What is Function Documentation?

When you write a function or method, you must describe:

- What it **expects** before running
- What it **does**
- What it **returns** afterward
- What issues or limits exist

This helps **any other developer (or future you)** understand the code easily.

3. Preconditions, Postconditions, and Invariants

These are the **rules** that define a function's behavior.

Term	Meaning	Example
Precondition	What must be true before the function runs	<code>open == true</code> before using <code>add()</code>
Postcondition	What must be true after it runs	<code>balance' == balance + anAmount</code>
Invariant	What never changes during execution	"balance must always be between -1000 and 1000"

💡 Simple view:

Pre = Before condition ✔️

Post = After condition ✔️

Invariant = Always true 🔄

4. Example Exercise

Suppose you have a class **Account** with:

- boolean `open`
- int `balance` (must stay between -1000 and +1000)
- Method `add(int anAmount)`

You must document this method.

✔️ Documentation Example:

Preconditions: `open == true`

`-1000 <= balance <= 1000`

`-1000 <= balance + anAmount <= 1000`

Postcondition: `balance' == balance + anAmount`

Returns: `balance'`

💡 This shows the method's **contract** — what it promises to do and under what conditions.

5. Describing How a Function Works

You can describe a function using:

- **Activity Diagrams (Flowcharts)** — visual representation
- **Pseudocode** — step-by-step logic in plain English

 Example:

“If the balance is valid, add the amount; else show an error.”

 These methods help visualize algorithms **before coding**, saving time and errors later.

6. Example: Address Book Flowchart

A flowchart might show how a simple address book program works:

- Ask user → “Add” or “Retrieve”?
- If Add → Enter name & address → Save to file
- If Retrieve → Ask name → Show address

 The diagram helps you (and others) understand the logic quickly.

7. Pseudocode Example: X-ray Controller

This example shows how pseudocode expresses logic **without syntax**:

FOR number of microseconds supplied by operator

 IF exceeds critical value

 try to get supervisor approval

 IF no approval → abort

 ENDIF

 IF power level too high → abort

 IF patient aligned & shield placed & self-test passed

 Apply X-ray

ENDFOR

 *It's readable like English — no programming syntax required.*

8. Pros & Cons of Pseudocode and Flowcharts

✓ Advantages

- Clarifies algorithms
- Catches design errors early
- Increases reliability
- Easier communication between team members

✗ Disadvantages

- Extra documentation to maintain
- Errors can occur while converting to code
- Sometimes need special tools for diagrams

💡 *But overall, these tools are worth using — especially in big projects.*

✂ Part 2 — Good Habits for Writing Functions

9. Good Habits (1/2)

Habit	Meaning	Example
Expressive names	Function & variable names should show purpose	Bad: <code>manipulate()</code> → Good: <code>getBaseRaisedToExponent()</code>
Avoid global variables	Pass parameters instead of using shared globals	Makes debugging easier
Defend against bad data	Check inputs & handle errors gracefully	Use conditions or exceptions

💡 Always write **self-explanatory** code — so it “tells a story” by itself.

10. Good Habits (2/2)

- Give **names to numbers** instead of using raw values
- `int NUM_CELLS = 8927;`

- `for (i = 0; i < NUM_CELLS; i++)`
- Limit parameters (ideally $\leq 6-7$)
- Initialize variables near first use
- Check loops for correctness and termination
- Avoid deep nesting (no more than 3 levels)
- Inspect your code before compiling

💡 These are like “clean code commandments” — they make your code readable and bug-free.

🌱 Part 3 — Software Design Basics

11. What is Software Design?

A **software design** is a complete plan or blueprint that guides programmers to write the code.

💡 Just like architects use blueprints for buildings, developers use **software design documents** to build programs.

It must be detailed enough that a developer can code directly from it — without confusion.

12. Goals of Software Design

Design turns **requirements (WHAT)** into a **solution (HOW)**.

It describes:

- The **system structure** (architecture)
- How components **fit together**
- How each part can be coded

🌱 Essentially: *Design bridges the gap between user needs and coding.*

13. Key Qualities (Goals) of a Good Design

Goal	Meaning
Correctness	Meets all requirements
Robustness	Handles errors gracefully
Flexibility	Easy to modify or expand
Reusability	Can be reused in other apps
Efficiency	Uses resources wisely
Reliability	Works consistently without failure
Usability	Easy for users to operate

🧠 These are the “7 pillars” of good design.

14. Explanation of Each Goal

● Correctness

The software must do exactly what was required — no more, no less.

● Robustness

It should survive unexpected conditions (invalid input, user mistakes, etc.)

Example: Program shouldn’t crash if user enters “abc” instead of a number.

● Flexibility

Easy to change or extend later.

Example: Adding a new feature like “withdraw” in a banking app shouldn’t break the system.

● Reusability

Write components that can be reused elsewhere.

Example: The same Customer class used in both billing and support systems.

● Efficiency

Use CPU and memory wisely — fast and lightweight software is always better.

● Reliability

It should rarely fail.

Measured by “average time between failures.”

Usability

Users should find it simple and intuitive to use.

Part 4 — Example: Command Line Calculator

15. Requirements

- Ask how many accounts to open (start with zero balance).
 - Let user choose which account to use.
 - Allow adding/subtracting money.
 - User can switch accounts or quit.
-

16. Implementation Issues

Problems often appear like:

- Invalid input → crash
- Wrong account number → crash
- Invalid string commands

 These show poor **robustness** and **flexibility**.

17. Design Improvement

You can improve by applying design goals:

- Add **error checking** (robustness)
 - Make it easy to **add features** (flexibility)
 - Make code **modular** and **reusable**
 - Ensure it runs fast (efficiency)
-

18. Summary

A **software design** is the complete blueprint for building software.

Goals Recap:

- Correctness
- Robustness
- Flexibility
- Reusability
- Efficiency
- Reliability
- Usability

In short:

“Good design means building software that works correctly, handles mistakes, adapts to change, and is easy to maintain.”

Would you like me to make a **summary sheet / short revision notes (2 pages)** for both lectures combined — ideal for exam revision (concepts + examples)?

Perfect  — now that you’ve uploaded all **three remaining lectures**, here’s the plan:
I’ll explain each —

- **Lecture 3: Object Orientation**
 - **Lecture 4: Design Principles I (Correctness & Robustness)**
 - **Lecture 5: Design Principles II (Flexibility, Reusability & Efficiency)**
in the **same clear, 80/20 understanding-based style** (like a teacher summarizing + explaining the real meaning of slides).
-

Lecture 3 — Object Orientation

1. What is Object Orientation?

Object Orientation (OO) is about **thinking in terms of real-world objects** — turning real-world things (like Customer, Account, Car) into **software objects** that contain:

- **Data** (attributes)
- **Behavior** (methods/functions)

 Example:

In a bank —

Customer, Account, and Transaction are all *objects* that interact.

2. Goal of Object Orientation

OO helps us describe software like we describe the **real world**:

“Customers Montague and Susan entered the bank and were served by teller Andy.”

Here:

- **Customer** → Class
 - **Montague, Susan** → Objects
 - **Teller** → Another object interacting with customers
-

3. Real-World Mapping

Every object in software should **represent something real**.

Example:

Real-world Concept Software Representation

Customer	class Customer {}
Account	class Account {}
Transaction	class Transaction {}

This **direct mapping** makes software easier to understand and maintain.

4. Benefits of Object Orientation

- ✓ Easier to design complex systems
 - ✓ Encourages code reuse
 - ✓ Easier maintenance
 - ✓ Models real-world relationships naturally
-

5. Classes & Objects

Term Meaning

Class Blueprint or template describing what an object is and what it can do.

Object Actual instance of a class (a real example).

 Example:

Class → Car

Object → myCar, toyotaCorolla2020

6. Members of a Class

Each class has:

- **Attributes (data):** e.g., mileage, vehicleID
- **Methods (behavior):** e.g., start(), stop()

Subclasses inherit these members too.

7. Types of Attributes

Type Meaning

Naming Unique for each object (e.g., vehicleID)

Descriptive Changes during object's life (e.g., mileage)

Referential Connects one class to another (e.g., owner of car)

8. Clients of a Class

A **client class** uses another class's methods.

Example:

```
class Customer { int computeBalance() { ... } }
```

```
class Bank {  
    void showAssets() {  
        Customer c = new Customer();  
        int balance = c.computeBalance();  
    }  
}
```

Here, Bank is a **client** of Customer.

9. Why OO is Useful

Object orientation supports:

- **Abstraction:** focus on essential details only
 - **Encapsulation:** hiding data within objects
 - **Inheritance:** reuse through hierarchy
 - **Polymorphism:** same action, different results depending on context
-

10. Example – Email Creation System

Different types of customers need different email messages (delinquent, regular, mountain).

OO allows you to create:

- Base class → Customer
- Subclasses → DelinquentCustomer, RegularCustomer, etc.

Each subclass defines its own message format — no messy if-else chains.

11. Problem with Non-OO Code

Using many if or switch conditions makes code:

- Hard to read
- Difficult to modify
- Expensive to maintain

OO solves this using **polymorphism**.

12. Polymorphism

Means “**many forms.**”

The same function name behaves differently for each subclass.

Example:

```
Customer c = new RegularCustomer();
```

```
c.sendEmail(); // sends regular promo
```

```
c = new DelinquentCustomer();
```

```
c.sendEmail(); // sends warning email
```

13. Interfaces

Interfaces group related methods and provide **clear contracts** for what a class should do.

Example:

```
interface Drawable {  
    void setColor(int color);  
    void draw();  
}
```

They help **organize complex classes** (like separating Pen, Color, Logo behaviors).

14. Summary

- **Class:** defines a concept
- **Object:** an instance of the class
- **Inheritance:** “is-a” relationship
- **Aggregation:** “has-a” relationship
- **Polymorphism:** same method, different behavior

 OO makes programs realistic, maintainable, and flexible.

Lecture 4 — Design Principles I: Correctness & Robustness

1. Design Principles Overview

Focuses on **two key principles**:

- **Correctness** – does it do what it’s supposed to?
 - **Robustness** – can it handle errors gracefully?
-

2. Correctness

A correct design **meets all requirements** exactly as specified.

Two ways to ensure correctness:

1. **Informal approaches** – logical reasoning, simplification, modular design
 2. **Formal approaches** – mathematical logic (rarely used outside critical systems)
-

3. Informal Correctness

Make designs:

- **Readable** (so they’re understandable)
- **Modular** (divide complex system into small parts)

 “If you can’t understand your design, you can’t claim it’s correct.”

4. Formal Correctness

Uses **invariants** — rules that remain true throughout program execution.

Example (Car class):

mileage > 0

mileage < 1,000,000

value >= -300

(type == "REGULAR" && value <= originalPrice) ||

(type == "VINTAGE" && value >= originalPrice)

These conditions define *correct states* of the object.

5. Using Invariants in Code

Keep variables **private** and change them only through **accessor methods** that preserve invariants.



Example: Setters that reject invalid data.

6. Interfaces and Modularity

- **Interface:** defines available functions (method signatures)
- **Modularity:** breaking design into clear, separate parts (classes/packages)

Interfaces make complex designs **manageable and testable**.

7. Domain vs. Non-Domain Classes

Type	Meaning	Example
Domain Classes	Specific to your application	Customer, Transaction
Non-Domain Classes	Generic helpers	Logger, DatabaseConnector

A good design uses both to keep code organized and reusable.

8. Robustness

The ability to handle unusual or wrong conditions **without crashing**.

Sources of errors:

- Invalid user input
 - Data communication errors
 - Developer mistakes
-

9. How to Improve Robustness

- ✓ Check input validity before using it
- ✓ Verify types and ranges
- ✓ Initialize variables properly
- ✓ Use exceptions or default values

Example:

```
if (aLength <= 0) throw new IllegalArgumentException("Invalid length");
```

10. Wrapping Parameters

Instead of passing raw data:

```
computeArea(int a, int b);
```

Use a class:

```
computeArea(Rectangle rect);
```

This ensures parameters are always valid (robustness + correctness).

11. Design Detail Balance

Too much design = slow progress

Too little = confusion later

 Rule: Simple tasks need little design, complex systems need full design documentation.

12. Summary

Principle	Meaning
-----------	---------

Correctness	Design must meet all requirements
--------------------	-----------------------------------

Robustness	Design must handle unexpected conditions gracefully
-------------------	---

Together, they make your system **reliable and trustworthy**.

Lecture 5 — Design Principles II: Flexibility, Reusability & Efficiency

1. Flexibility

Ability to **easily change or extend** software.

 Example: In a banking system, adding a new type of account shouldn't break existing code.

Ways to achieve flexibility:

- Use **abstract classes** or **interfaces**
- Use **inheritance** for future extensions
- Avoid “hard-coding” things that may change

2. Reusability

The ability to use existing code **in new contexts** without modification.

Tips:

- ✓ Write methods independent of specific classes (use static if possible)
- ✓ Keep methods well-documented and parameterized
- ✓ Use expressive, meaningful names
- ✓ Reduce dependencies between classes

3. Improving Class Reusability

- Make class represent a real-world concept

- Reduce unnecessary links to other classes
- Keep it general enough to fit future needs

 Example: Instead of OnlineStudent, create Student and reuse for online or campus.

4. Efficiency

Means **using time and memory wisely** — without sacrificing flexibility.

Two dimensions:

- **Time efficiency:** how fast it runs
- **Space efficiency:** how much memory it uses

 Always balance efficiency with readability and maintainability.

5. Trade-offs

Sometimes you must **compromise**:

Design Goal	Possible Trade-off
-------------	--------------------

More flexibility	May reduce speed
------------------	------------------

More reusability	May require extra memory
------------------	--------------------------

More efficiency	May reduce clarity
-----------------	--------------------

6. Practical Techniques

- Use **profilers** to detect performance bottlenecks
 - Reduce **loop complexity**
 - Minimize **network calls**
 - Store only essential data
-

7. Example – Calculator Design

Flexible design:

- Separate logic into **classes (Add, Multiply, Divide)**
 - Each operation extends base Calculator class
→ Easy to add new operations later.
-

8. Summary

Principle Meaning

Flexibility Easy to change or extend

Reusability Useful across multiple projects

Efficiency Performs well in time and space

 **Good design balances all three** — flexible enough to change, reusable for other work, and efficient enough to perform well.

Would you like me to make a **combined “Final Revision Notes” file (PDF or DOCX)** — with summarized points, examples, and diagrams from **all five lectures** (Prologue → Design Principles II) in one easy-to-study format?